API REFERENCE DOCUMENTATION

Serva Encoder WebApp Platform

Complete Endpoint Reference | Request & Response Schemas | Authorization

VERSION 1.2

Overview

PROPERTY	VALUE
Version	1.2
Base URL	`http://5.78.120.15`
Auth Service	Port 8001 internally, proxied via nginx at `/api/auth/`
Backend	Port 8000 internally, proxied via nginx at `/api/`
Protocol	HTTPS only — TLS 1.3. All HTTP connections are rejected (444).
Format	JSON request and response bodies. UTF-8 encoding.
Billing Model	Beta tier: 1TB/month free. Premium = unlimited.

Endpoint Index

#	METHOD	ENDPOINT	SERVICE	DESCRIPTION	AUTH REQUIRED
1	POST	`/auth/google/callback`	Auth	Google OAuth login	None
2	POST	`/auth/logout`	Auth	Revoke session token	Bearer
3	POST	`/keys`	Auth	Generate Service API Key	Bearer
4	GET	`/keys`	Auth	List user's API Keys	Bearer
5	DELETE	`/keys/{key_id}`	Auth	Revoke API Key	Bearer
6	GET	`/quota`	Auth	Get monthly quota & usage	Bearer
7	POST	`/api/encode`	Auth	Initiate file encoding	Bearer
8	POST	`/api/decode`	Auth	Initiate file decoding	Bearer
9	POST	`/api/stripe/webhook`	Auth	Stripe webhook handler	Stripe Signature
10	GET	`/health`	Auth	Auth Service health check	None
11	POST	`/api/stream/{file_reference}`	Backend	Stream file for encoding	Streaming Token
12	POST	`/api/decode`	Backend	Stream .serva for decoding	Streaming Token
13	GET	`/download/{file_id}`	Backend	Download encoded/decoded file	None
14	GET	`/api/stats/public`	Backend	Platform aggregate statistics	None
15	GET	`/api/stats/extensions`	Backend	File extension frequencies	None
16	GET	`/health`	Backend	Backend health check	None

Authorization Overview

The API accepts three bearer credential types. The `get_current_user` dependency detects the type by format and routes accordingly.

AUTH TYPE	FORMAT	TTL	DETECTION	USED FOR
Session Token	`sess_<urlsafe-64>`	24 hours	`sess_` prefix → Redis lookup	All authenticated endpoints (recommended)
Google ID Token	JWT (3 base64 segments)	~1 hour	2 dots → Google tokeninfo API	Backward compatible; re-authenticate on expiry
Service API Key	`sk_live_<128-hex>`	Never (until revoked)	Else → HMAC-SHA256 validation	Programmatic / server-to-server access
Streaming Token	Random urlsafe string	300 seconds	`X-Streaming-Token` header	Single-use file streaming only
Stripe Signature	HMAC-SHA256	Per event	`Stripe-Signature` header	Stripe webhook only

Recommended flow: Call `POST /auth/google/callback`, store the returned `session_token`, and send `Authorization: Bearer <session_token>` on all subsequent requests. Revoke it on logout via `POST /auth/logout`.

Security note: Service API Keys are stored as `HMAC-SHA256(key, server_pepper)` and compared using constant-time `hmac.compare_digest`. Streaming tokens are single-use and deleted from Redis immediately on first consumption.

Section 1 — Authentication Endpoints

POST /auth/google/callback

Exchange Google ID token for user profile and session token

Authorization Required: None — this is the login endpoint.

Description

Authenticates a user via Google OAuth 2.0. Exchanges the Google ID token for a user profile and a server-issued session token. Creates a new account automatically on first login. The returned `session_token` should be stored and used as the `Authorization: Bearer` header for all subsequent requests. It expires after 24 hours (configurable via `SESSION_TOKEN_TTL_SECONDS`).

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`token`	string	Yes	Google OAuth ID token (JWT) from the Google Sign-In client-side flow.

Example Request

```
POST /auth/google/callback
Content-Type: application/json

{
  "token": "eyJhbGciOiJSUzI1NiIsImtpZCI6IjFlOWdkazcifQ..."
}
```

Responses

STATUS	MEANING	RESPONSE BODY
200	Success	`{ "user_id", "email", "google_id", "plan_type", "subscription_status", "beta_tier_active", "beta_enrolled_at", "created_at", "session_token": "sess_..." }`
401	Invalid Token	`{ "detail": "Invalid or expired Google OAuth token" }`
401	Unverified Email	`{ "detail": "Google account email is not verified" }`

POST /auth/logout

Revoke the current session token

Authorization Required: Bearer <session_token | google_id_token | api_key>

Description

Immediately revokes the current session token by deleting it from Redis. No-op if called with a Google ID token or Service API Key — those have their own expiry/revocation mechanisms.

Example Request

```
POST /auth/logout
Authorization: Bearer sess_Rn4kLmP9xQs7vW2...
```

Responses

STATUS	MEANING	RESPONSE BODY
204	Revoked	No content
401	Unauthorized	`{ "detail": "Invalid or expired session token" }`

Section 2 — API Key Endpoints

All endpoints require `Authorization: Bearer <session_token | google_id_token | api_key>`.

POST /keys

Generate a new Service API Key

Authorization Required: Bearer <session_token | google_id_token | api_key>

Description

Generates a new Service API Key for the authenticated user. The `raw_key` is returned **once only** and is never stored — only the `HMAC-SHA256(key, server_pepper)` hash is persisted. Store the key immediately and securely.

⚠ The raw API key is returned only once. If lost, the user must generate a new key.

Example Request

```
POST /keys
Authorization: Bearer sess_Rn4kLmP9xQs7vW2...
```

Responses

STATUS	MEANING	RESPONSE BODY
201	Created	`{ "key_id": "uuid", "key_prefix": "sk_live_ab12", "raw_key": "sk_live_...", "created_at": "..." }`
401	Unauthorized	`{ "detail": "Invalid or expired session token" }`

GET /keys

List all API keys for the authenticated user

Authorization Required: Bearer <session_token | google_id_token | api_key>

Description

Returns all API keys for the authenticated user, ordered by creation date descending. Raw keys are **never** returned after initial creation. Response is a JSON array.

Example Request

```
GET /keys
Authorization: Bearer sess_Rn4kLmP9xQs7vW2...
```

Success Response — 200 OK

```
[
  {
    "key_id": "3f7a9c2e-1b4d-4e8f-a3c1-9d2e7f8b4a0c",
    "key_prefix": "sk_live_ab12",
    "revoked": false,
    "created_at": "2025-11-01T00:00:00Z",
    "last_used_at": "2025-11-15T14:32:00Z"
```

```
    }
  ]
```

Responses

STATUS	MEANING	RESPONSE BODY
200	Success	Array of API key objects (see above)
401	Unauthorized	`{ "detail": "Invalid or expired session token" }`

DELETE /keys/{key_id}

Permanently revoke an API key

Authorization Required: Bearer <session_token | google_id_token | api_key>. Users can only revoke their own keys.

Description

Permanently revokes a Service API Key. Sets `revoked=true` in the database and immediately invalidates the Redis cache. Cannot be undone.

Path Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`key_id`	UUID	Yes	The `key_id` returned when the API key was created. UUID v4, 36 characters.

Example Request

```
DELETE /keys/3f7a9c2e-1b4d-4e8f-a3c1-9d2e7f8b4a0c
Authorization: Bearer sess_Rn4kLmP9xQs7vW2...
```

Responses

STATUS	MEANING	RESPONSE BODY
204	Revoked	No content
401	Unauthorized	`{ "detail": "Invalid or expired session token" }`
404	Not Found	`{ "detail": "API key not found or does not belong to you" }`

Section 3 — Quota Endpoint

GET **/quota**

Get current month's usage and quota

Authorization Required: Bearer <session_token | google_id_token | api_key>

Description

Returns the authenticated user's current plan type, monthly usage, and quota. For **Free** plan: quota is 1TB/month. For **Premium** plan: `quota_bytes`, `quota_remaining_bytes`, and `percentage_used` are `null` (unlimited).

Example Request

```
GET /quota
Authorization: Bearer sess_Rn4kLmP9xQs7vW2...
```

Success Response — 200 OK

```
{
  "plan_type": "free",
  "total_bytes_this_month": 107374182400,
  "quota_bytes": 1099511627776,
  "quota_remaining_bytes": 992137445376,
  "percentage_used": 9.765625
}
```

Responses

STATUS	MEANING	RESPONSE BODY
200	Success	Quota and usage object (see above)
401	Unauthorized	`{ "detail": "Invalid or expired session token" }`

Section 4 — Encoding Endpoints

Encoding is a three-step flow:

1. `POST /api/encode` on the **Auth Service** — validates quota, returns `streaming_token`
2. `POST /api/stream/{file_reference}` on the **Backend** — streams and encodes the file
3. `GET /download/{file_id}` on the **Backend** — downloads the `.serva` result

POST /api/encode — Auth Service

Step 1: Initiate file encoding — validates quota and returns streaming token

Authorization Required: Bearer <session_token | google_id_token | api_key>

Description

Validates credentials, checks monthly quota, generates a single-use streaming token bound to `file_reference` + `user_id`, forwards the job to the Backend, and returns the streaming token to the client. After receiving the response, the browser must immediately call `POST /api/stream/{file_reference}` on the Backend.

> `user_password` is forwarded opaquely to the Backend — never logged, stored in the database, or inspected by the Auth Service.

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`idempotency_key`	UUID string	Yes	UUID v4. Prevents duplicate encoding within a 10-minute window, scoped per user.
`file_reference`	UUID string	Yes	UUID v4 identifier for the file held in browser memory.
`file_size_bytes`	integer	Yes	Declared file size in bytes. Validated server-side against actual bytes received.
`file_extension`	string	No	Original file extension (e.g. `.pdf`). Default `""`. Max 10 chars.
`original_filename`	string	No	Original filename. Embedded in `.serva` header for decode recovery. Max 512 chars.
`user_password`	string	Yes	AES-256-GCM encryption password. Ephemeral — never stored or logged.

File size limits: Free plan = 10GB per file. Premium plan = 1TB per file.

Example Request

```
POST /api/encode
Authorization: Bearer sess_Rn4kLmP9xQs7vW2...
Content-Type: application/json

{
  "idempotency_key": "a1b2c3d4-5e6f-7a8b-9c0d-1e2f3a4b5c6d",
  "file_reference": "3f7a9c2e-1b4d-4e8f-a3c1-9d2e7f8b4a0c",
  "file_size_bytes": 10485760,
  "file_extension": ".pdf",
  "original_filename": "report.pdf",
  "user_password": "my-secure-password"
}
```

Success Response — 200 OK

```
{
  "file_id": "9a1b3c5d-2e4f-6a8b-0c2d-4e6f8a0b2c4d",
  "download_url": "/download/9a1b3c5d-2e4f-6a8b-0c2d-4e6f8a0b2c4d",
```

```
  "streaming_token": "K3mL9pQs7vW2xR4t..."
}
```

Error Responses

STATUS	MEANING	RESPONSE BODY
200	Success	`{ "file_id", "download_url", "streaming_token" }`
400	File too large	`{ "detail": "File size exceeds 10GB limit for free plan" }`
401	Unauthorized	`{ "detail": "Invalid or expired session token" }`
409	Duplicate	`{ "error": "duplicate_request", "message": "...", "status": "processing", "original_request_id": "uuid" }`
410	Already downloaded	`{ "detail": "File already downloaded — use a new idempotency_key" }`
429	Quota exceeded	`{ "detail": "Monthly encoding quota exceeded (Free plan: 1TB/month)." }`
502	Backend down	`{ "detail": "Backend service unavailable" }`

POST /api/stream/{file_reference} — Backend

Step 2: Stream raw file bytes from browser for encoding

Authorization Required: X-Streaming-Token header — single-use, 300s TTL, from POST /api/encode response.

Description

Receives the raw file stream from the browser, runs the Serva encoding algorithm, encrypts with AES-256-GCM, and stores the result at `/tmp/encoded/{file_id}.serva`. The streaming token is validated and deleted on first use. Byte count (declared vs received) is validated **before** encryption to prevent quota manipulation.

Path Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`file_reference`	UUID	Yes	Must match the `file_reference` bound to the streaming token.

Request Headers

HEADER	REQUIRED	DESCRIPTION
`X-Streaming-Token`	Yes	Single-use token from `POST /api/encode` response.

Request Body: Raw binary file bytes — streamed directly, no multipart wrapper.

Success Response — 200 OK

```
{
  "status": "complete",
  "file_id": "9a1b3c5d-2e4f-6a8b-0c2d-4e6f8a0b2c4d",
  "original_size_bytes": 10485760,
  "encoded_size_bytes": 8388608,
  "file_size_bytes": 8388640
}
```

Error Responses

STATUS	MEANING	RESPONSE BODY
400	Empty file	`{ "detail": "Empty file received" }`
400	Byte mismatch	`{ "error": "byte_count_mismatch", "declared": 10485760, "received": 10485700 }`
401	Invalid token	`{ "detail": "Invalid, expired, or already-used streaming token" }`
408	Disconnected	`{ "detail": "Browser disconnected during upload" }`
500	Encode failure	`{ "detail": "Encoding or encryption failure" }`
501	Unsupported type	`{ "detail": "Unsupported file type for encoding" }`

Section 5 — Decoding Endpoints

Decoding is a three-step flow and is **free** (no quota check):

1. `POST /api/decode` on the **Auth Service** — returns a streaming token with the embedded password
2. `POST /api/decode` on the **Backend** — streams, decrypts, and decodes the `.serva` file
3. `GET /download/{file_id}` on the **Backend** — downloads the original file

POST /api/decode — Auth Service

Step 1: Initiate file decoding — returns streaming token with embedded password

Authorization Required: Bearer <session_token | google_id_token | api_key>

Description

Validates credentials, creates a decode streaming token with the password embedded in Redis (TTL 300s), and returns the token to the browser. Decoding is free — no quota check is performed. The browser must immediately call `POST /api/decode` on the Backend with this token.

> `user_password` is stored transiently in Redis inside the streaming token. It is extracted by the Backend and cleared after decryption — **never** sent in the request body to the Backend.

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`file_reference`	UUID string	Yes	UUID v4 identifier for the `.serva` file held in browser memory.
`file_size_bytes`	integer	Yes	Size of the `.serva` file in bytes. Validated by Backend before decryption.
`user_password`	string	Yes	Decryption password. Ephemeral — never logged, stored, or inspected.

Example Request

```
POST /api/decode
Authorization: Bearer sess_Rn4kLmP9xQs7vW2...
Content-Type: application/json

{
  "file_reference": "3f7a9c2e-1b4d-4e8f-a3c1-9d2e7f8b4a0c",
  "file_size_bytes": 8388640,
  "user_password": "my-secure-password"
}
```

Success Response — 200 OK

```
{
  "streaming_token": "N7pR2sT5wX8yZ1q...",
  "file_reference": "3f7a9c2e-1b4d-4e8f-a3c1-9d2e7f8b4a0c"
}
```

Error Responses

STATUS	MEANING	RESPONSE BODY
200	Success	`{ "streaming_token", "file_reference" }`
401	Unauthorized	`{ "detail": "Invalid or expired session token" }`

500	Redis error	`{ "detail": "Internal server error" }`

POST /api/decode — Backend

Step 2: Stream .serva bytes from browser for decryption and decoding

Authorization Required: X-Streaming-Token header — single-use, 300s TTL, from Auth Service POST /api/decode response.

Description

Receives the `.serva` file stream from the browser. Validates the streaming token, extracts the password from Redis, validates byte count **before** decryption, decrypts with AES-256-GCM, and Serva-decodes back to the original format. Stores the result at `/tmp/decoded/{file_id}`.

Request Headers

HEADER	REQUIRED	DESCRIPTION
`X-Streaming-Token`	Yes	Single-use token from Auth Service `POST /api/decode` response.

Request Body: Raw binary `.serva` file bytes — streamed directly, no multipart wrapper.

Success Response — 200 OK

```
{
  "file_id": "a3f8b924-1c5d-4e7f-9012-bc3d4e5f6a7b",
  "original_filename": "report.pdf",
  "file_size_bytes": 10485760,
  "download_url": "/download/a3f8b924-1c5d-4e7f-9012-bc3d4e5f6a7b"
}
```

Error Responses

STATUS	MEANING	RESPONSE BODY
400	Empty file	`{ "detail": "Empty file received" }`
400	Byte mismatch	`{ "detail": "Byte count mismatch" }`
400	Invalid file	`{ "detail": "Invalid .serva file structure" }`
401	Invalid token	`{ "detail": "Invalid, expired, or already-used streaming token" }`
401	Wrong password	`{ "detail": "Incorrect password or corrupted file" }`
408	Disconnected	`{ "detail": "Browser disconnected during upload" }`
500	Decode failure	`{ "detail": "Decryption or decoding failure" }`

Section 6 — Download Endpoint

GET /download/{file_id} — Backend

Download encoded or decoded file — file is deleted immediately after download

Authorization Required: None — file_id is ephemeral and single-use.

Description

Downloads the result file. **The file is permanently and immediately deleted from the server after this download.** Checks `/tmp/encoded/{file_id}.serva` first, then `/tmp/decoded/{file_id}`.

Path Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`file_id`	UUID	Yes	File ID returned by the stream or decode endpoint.

Query Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`filename`	string	No	Override the suggested download filename.

Response Headers (200)

HEADER	DESCRIPTION
`Content-Type`	`application/octet-stream`
`Content-Length`	File size in bytes
`X-File-Id`	The file UUID

Error Responses

STATUS	MEANING	RESPONSE BODY
200	Success	Binary file stream
404	Not Found	`{ "detail": "File not found or already downloaded" }`
409	Not Ready	`{ "detail": "Encoding not yet complete" }`

Section 7 — Public Statistics Endpoints

GET /api/stats/public — Backend

Platform-wide aggregate statistics

Authorization Required: None — completely public endpoint.

Description

Returns platform-wide aggregate statistics. No authentication required. Used for public landing page display. Updated in real-time after each successful encoding operation.

Example Request

```
GET /api/stats/public
```

Success Response — 200 OK

```
{
  "platform_total_input_bytes": 523599479111680,
  "platform_total_output_bytes": 104719895822336,
  "platform_total_savings_bytes": 418879583289344,
  "platform_average_compression_ratio": 0.80,
  "platform_total_files_encoded": 52000
}
```

Responses

STATUS	MEANING	RESPONSE BODY
200	Success	Platform aggregate statistics (see above)
500	Server Error	`{ "detail": "Failed to retrieve statistics" }`

GET /api/stats/extensions — Backend

All file extension frequencies, sorted by count descending

Authorization Required: None — completely public endpoint.

Description

Returns frequency counts for all file extensions processed on the platform. Sorted by count descending. Used for public landing page display.

Example Request

```
GET /api/stats/extensions
```

Success Response — 200 OK

```
{
  "extensions": [
    { "extension": ".jpg", "count": 18200, "percent_of_total": 35.0 },
    { "extension": ".png", "count": 10400, "percent_of_total": 20.0 },
    { "extension": ".pdf", "count":  5200, "percent_of_total": 10.0 }
  ],
```

```
  "total_files_encoded": 52000
}
```

Responses

STATUS	MEANING	RESPONSE BODY
200	Success	All file extensions with counts and percentages
500	Server Error	`{ "detail": "Failed to retrieve extension statistics" }`

Section 8 — Billing Endpoints

POST /api/stripe/webhook — Auth Service

Receive and process Stripe subscription lifecycle events

Authorization Required: Stripe-Signature header — validated using STRIPE_WEBHOOK_SECRET. Do NOT call directly.

Description

Receives webhook events from Stripe for subscription management. Verifies the `Stripe-Signature` header. All plan changes are written to the audit log and invalidate the Redis plan cache immediately.

Handled Stripe Events

STRIPE EVENT	EFFECT
`customer.subscription.created`	Sets `plan_type = "premium"`, `subscription_status = "active"`. Writes audit log. Invalidates Redis cache.
`customer.subscription.deleted`	Sets `plan_type = "free"`, `subscription_status = "canceled"`. Writes audit log. Invalidates Redis cache.
`invoice.payment_failed`	Sets `subscription_status = "past_due"`. Writes audit log. Invalidates Redis cache.

Responses

STATUS	MEANING	RESPONSE BODY
200	Processed	`{ "status": "ok" }`
400	Invalid Signature	`{ "detail": "Invalid Stripe webhook signature" }`

Section 9 — Health Check Endpoints

GET /health — Auth Service (port 8001)

Auth Service health check

Authorization Required: None — used by Docker Compose health monitoring.

Responses

STATUS	MEANING	RESPONSE BODY
200	Healthy	`{ "status": "healthy", "service": "auth" }`

GET /health — Backend (port 8000)

Backend health check with active job and file counts

Authorization Required: None — used by Docker Compose health monitoring.

Responses

STATUS	MEANING	RESPONSE BODY
200	Healthy	`{ "status": "healthy", "service": "backend", "active_jobs": 3, "encoded_files": 12 }`

Section 10 — Error Codes & Troubleshooting

All error responses use FastAPI's standard structure:

```
{ "detail": "Human-readable error message" }
```

STATUS	ERROR	CAUSE	RESOLUTION
400	`validation_error`	Request field failed Pydantic validation	Check field types, lengths, and formats per this spec
400	`byte_count_mismatch`	Received bytes ≠ declared `file_size_bytes`	Ensure declared size matches actual file size
400	`invalid_serva_file`	Invalid `.serva` file structure	Verify the file is a valid `.serva` encoded file
401	`invalid_session_token`	Session token expired, invalid, or revoked	Re-authenticate via `POST /auth/google/callback`
401	`invalid_google_token`	Google ID token expired or invalid	Re-authenticate via Google OAuth
401	`invalid_api_key`	API key missing, invalid, or revoked	Verify key starts with `sk_live_` ; regenerate if revoked
401	`invalid_token`	Streaming token invalid, expired, or already consumed	Retry `POST /api/encode` or `POST /api/decode` for a fresh token
401	`incorrect_password`	Wrong password — AES-256-GCM `InvalidTag`	Verify the password used during original encoding
404	`not_found`	Resource does not exist or was already downloaded	File is single-download only; re-encode if needed
408	`stream_timeout`	Browser disconnected during file streaming	Keep browser open during encoding; retry the full flow
409	`duplicate_request`	Idempotency key already in progress	Wait for original to complete or use a new `idempotency_key`
409	`encoding_in_progress`	Encoding not yet complete	Poll `GET /download/{file_id}` with a short delay
410	`file_already_downloaded`	File already downloaded for this `idempotency_key`	Submit a new encode request with a new `idempotency_key`
429	`quota_exceeded`	Free plan 1TB/month quota exceeded	Upgrade to Premium for unlimited encoding
500	`internal_error`	Unexpected server error	Retry after 60s; contact support if persistent
501	`not_implemented`	Unsupported file type for encoding	Check supported file types
502	`backend_unavailable`	Backend unreachable from Auth Service	Check Backend service health at `GET /health`
503	`service_unavailable`	Service temporarily unavailable	Retry with exponential backoff

All responses include the standard FastAPI error format. Rate-limited responses include a `Retry-After` header in seconds.